Контроль качества данных по целям сотрудников

Как работает контроль качества данных

Сейчас (AS IS). Система рассчитывает технические метрики, но руководителям не хватает наглядной информации о качестве данных по целям и ключевым результатам (КР).

Целевой процесс (TO BE). Руководители и администраторы системы планируют проверять данные еженедельно. Модуль контроля качества данных (ККД) использует существующие данные в PostgreSQL. За один запуск проверки (прогон) модуль применяет правила к целям и КР, регистрирует проблемы и рассчитывает метрики с учётом каскада целей. Сервисы на Java передают результаты на страницу ККД через API.

Задачи пользователей

- Руководитель видит метрики по целям и КР своего подразделения, включая дочерние подразделения, и может посмотреть, какие проблемы повлияли на результат.
- При повторной проверке пользователь видит, как изменились метрики и какие проблемы появились, сохранились или исчезли.
- Разработчик использует модель данных, правила и контракт API для реализации модуля и проверки его работы.

1. Модель данных

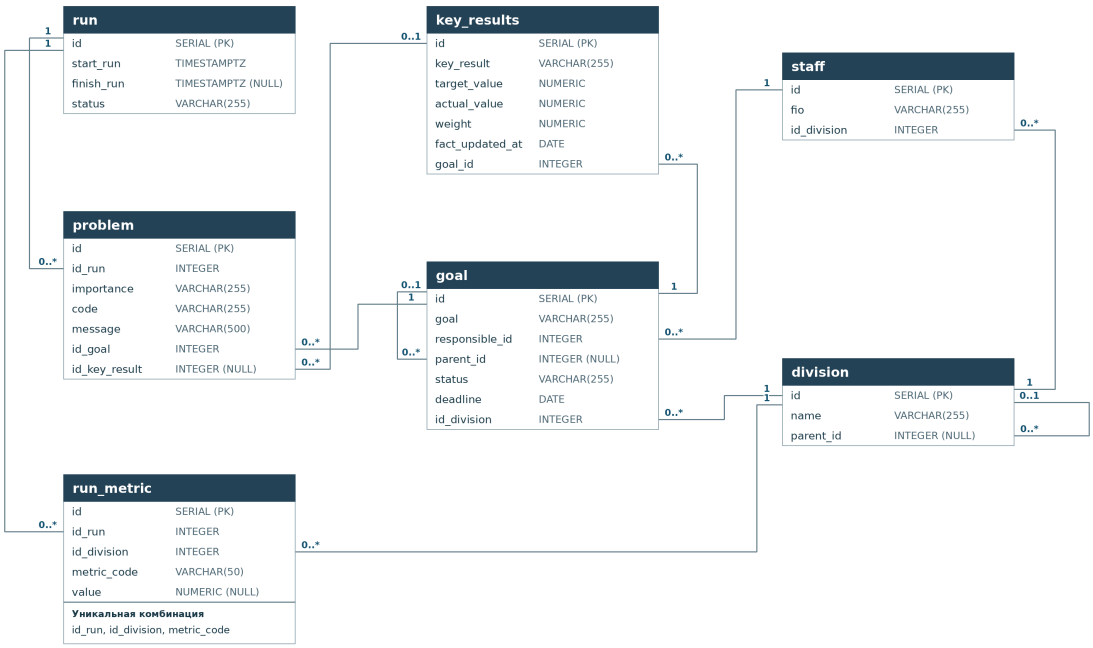


Рисунок 1. Модель данных ККД.

`division` — подразделение оргструктуры

Каждая запись представляет подразделение. Связь с вышестоящим подразделением задаёт поле `parent_id`.

Поле	Тип	Описание
id	SERIAL (PK)	Уникальный номер подразделения. На него ссылаются другие таблицы (goal.id_division, staff.id_division, дочерние division.parent_id).
name	VARCHAR (255)	Название подразделения
parent_id	INTEGER (NULL)	Идентификатор родительского подразделения (`division.id`). У корневого подразделения — `NULL`.

`goal` — цель сотрудника / подразделения

Каждая цель закреплена за подразделением и ответственным сотрудником.

Поле	Тип	Описание
id	SERIAL (PK)	Уникальный номер цели. На него ссылаются КР (key_results.goal_id), дочерние цели (parent_id) и проблемы (problem.id_goal).
goal	VARCHAR (255)	Текст формулировки цели, например «Вырастить выручку».
responsible_id	INTEGER	Идентификатор сотрудника, ответственного за цель (`staff.id`).
parent_id	INTEGER (NULL)	Идентификатор родительской цели (`goal.id`). У корневой цели — `NULL`.
status	VARCHAR (255)	Статус цели: `active` (в работе), `finished` (завершена), `cancelled` (отменена). При создании цель получает статус `active`.
deadline	DATE	Плановая дата завершения цели.
id_division	INTEGER	Идентификатор подразделения, к которому относится цель (`division.id`).

`key_results` — ключевой результат (КР)

Каждый КР измеряет прогресс по одной цели.

Поле	Тип	Описание
id	SERIAL (PK)	Уникальный номер ключевого результата. На него ссылается problem.id_key_result.
key_result	VARCHAR(255)	Формулировка ключевого результата, например «Сократить время обработки заявки».
target_value	NUMERIC	Значение показателя, которого планируют достичь.
actual_value	NUMERIC	Фактически достигнутое значение показателя. При создании КР задаётся текущее значение (например 0). Далее обновляется пользователем вместе с датой обновления fact_updated_at.
weight	NUMERIC	Вес вклада КР в цель в процентах. Вес каждого КР должен быть от 5 до 100 включительно, а сумма всех весов КР цели должна равняться 100.

Поле	Тип	Описание
fact_updated_at	DATE	Дата последнего обновления actual_value. При создании КР указывается дата создания. Поле обязательно при вводе. Используется в правиле `KR_NO_UPDATE`.
goal_id	INTEGER	Номер цели, которой принадлежит КР (goal.id).

`staff` — сотрудник

Сотрудник относится к подразделению и может отвечать за выполнение цели.

Поле	Тип	Описание
id	SERIAL (PK)	Уникальный номер сотрудника. На него ссылается goal.responsible_id.
fio	VARCHAR (255)	Фамилия, имя, отчество (или полное отображаемое имя) сотрудника.
id_division	INTEGER	Номер подразделения, в котором числится сотрудник (division.id).

`run` — один запуск (прогон) проверки ККД

Каждый запуск проверки регистрируется отдельно, в том числе при ошибке.

Поле	Тип	Описание
id	SERIAL (PK)	Уникальный номер прогона. На него ссылаются проблемы (problem.id_run) и значения метрик (run_metric.id_run).
start_run	TIMESTAMPTZ	Дата и время начала выполнения проверки.
finish_run	TIMESTAMPTZ (NULL)	Дата и время окончания проверки. Если прогон ещё выполняется (running), то NULL.
status	VARCHAR (255)	Состояние проверки: `running` — выполняется, `success` — успешно завершена, `error` — завершилась с ошибкой.

`problem` — найденная проблема качества данных

Запись связывает нарушение с правилом, объектом проверки и прогоном.

Поле	Тип	Описание
id	SERIAL (PK)	Уникальный номер записи о найденной проблеме.
id_run	INTEGER	Номер прогона, в котором нашли проблему (run.id). Связывает находку с конкретным запуском ККД.
importance	VARCHAR (255)	Уровень важности проблемы для приоритизации на экране (high/medium/low).
code	VARCHAR (255)	Стабильный код сработавшего правила (GOAL_STATUS_STALE, GOAL_BAD_WEIGHTS, GOAL_STATUS_CONFLICT, KR_NO_UPDATE).
message	VARCHAR (500)	Текст проблемы, который отображается пользователю (например, «Сумма весов КР цели равна 80, а должна быть 100»).
id_goal	INTEGER	Номер цели, к которой относится проблема. Заполняется всегда: для правил по цели — сама цель, для правил по КР — цель этого КР (`key_results.goal_id`).

Поле	Тип	Описание
id_key_result	INTEGER (NULL)	Номер КР, если проблема по ключевому результату (`KR_NO_UPDATE`). Если проблема только по цели — `NULL`. По этому полю отличают тип объекта: `id_key_result IS NOT NULL` — проблема КР, `id_key_result IS NULL` — проблема цели.

`run_metric` — значение метрики по итогам прогона

Таблица хранит значения метрик по подразделениям на момент успешного прогона.

Поле	Тип	Описание
id	SERIAL (PK)	Уникальный номер записи.
id_run	INTEGER	Прогон (`run.id`), в котором посчитали метрику.
id_division	INTEGER	Подразделение (`division.id`), для ветки которого посчитана метрика.
metric_code	VARCHAR (50)	Код метрики: `affected_goals_pct`, `stale_kr_pct`, `cascade_conflict_pct`.
value	NUMERIC (NULL)	Значение метрики на момент прогона. `NULL`, если посчитать нельзя (нет активных целей / нет КР / нет завершённых или отменённых родительских целей — по правилам конкретной метрики).

2. Правила проверки целей и ключевых результатов

Для всех правил активной считается цель с `goal.status = 'active'`.

Правило 1 — Просроченные активные цели

Поиск активных целей с неактуальными сроками. Если на дату проверки срок уже прошёл, а цель всё ещё в работе, система регистрирует проблему.

Поле	Значение
Код	GOAL_STATUS_STALE
Объект проверки	Цель (`goal`)
Важность	high
Условие нарушения	`deadline < CURRENT_DATE AND status = 'active'`.

Текст сообщения: «Срок цели истёк {deadline}. Проверьте её статус: отмените или завершите цель либо перенесите срок».

Пример: У активной цели срок 5 ноября. При проверке 5 ноября нарушения нет. Если 6 ноября статус всё ещё `active`, цель попадёт в список проблем. Если цель завершили, отменили или перенесли срок на 10 ноября, проверка 6 ноября пройдёт.

Правило 2 — Некорректные веса ключевых результатов цели

У активной цели должен быть хотя бы один ключевой результат. Вес каждого КР — от 5 до 100% включительно. Сумма весов всех КР цели — 100%.

Поле	Значение
Код	GOAL_BAD_WEIGHTS
Объект проверки	Цель (`goal`)
Важность	high
Условие нарушения	Выполнено хотя бы одно условие: у активной цели нет КР, вес какого-либо КР не заполнен или выходит за пределы 5–100%, либо сумма весов отличается от 100%.

Текст сообщения: Для цели без КР — «У активной цели нет ключевых результатов». Если у цели есть КР, в сообщении перечисляются обнаруженные нарушения: «У КР № {id} не указан вес», «Вес КР № {id} равен {weight}%. Допустимый диапазон — от 5 до 100%», «Сумма весов равна {sum}%. Требуется 100%». Сумма указывается, когда заполнены все веса.

Пример: У активной цели № 20 два КР с весами 4% и 96%. В сумме они дают 100%, но первый вес ниже минимума. Проблема регистрируется для цели № 20. Если изменить веса на 5% и 95%, проверка пройдёт.

```
SELECT g.id
FROM goal g
LEFT JOIN key_results kr ON kr.goal_id = g.id
WHERE g.status = 'active'
GROUP BY g.id
HAVING COALESCE(SUM(kr.weight), 0) <> 100
    OR COUNT(kr.id) <> COUNT(kr.weight)
    OR MIN(kr.weight) < 5
    OR MAX(kr.weight) > 100;
```

Правило 3 — Активные потомки завершённых или отменённых целей

Каскад целей проверяется сверху вниз по связям `goal.parent_id`. Если родительскую цель завершили или отменили, а в её ветке ещё есть активная цель, проблема регистрируется у родительской цели.

Поле	Значение
Код	GOAL_STATUS_CONFLICT
Объект проверки	Родительская цель (`goal`)
Важность	high
Условие нарушения	Статус родительской цели — `finished` или `cancelled`. Среди её потомков на любом уровне есть хотя бы одна цель со статусом `active`. За прогон создаётся одна запись `problem` на такую родительскую цель, её номер записывается в `id_goal`.

Текст сообщения: «Цель № {id} {статус цели}, но в её ветке остаются активные цели. Проверьте статус цели № {id}». Для `finished` подставляется «завершена», для `cancelled` — «отменена».

Пример: Цель № 10 завершена, её дочерняя цель № 11 отменена, а цель № 12 под целью № 11 всё ещё активна. Проблема регистрируется у цели № 11 и у цели № 10: у обеих в ветке есть активная цель № 12.

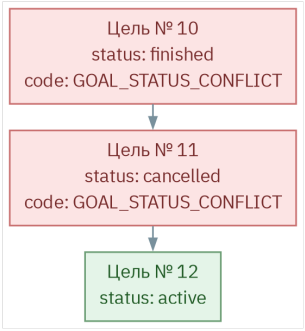


Рисунок 2. Иллюстрация примера правила 3.

Правило 4 — Устаревшие фактические значения ключевых результатов

У ключевых результатов активной цели проверяется дата последнего обновления фактического значения. Допустимый интервал — 28 календарных дней (четыре недели). При создании КР фактическое значение и дата его обновления обязательны.

Поле	Значение
Код	KR_NO_UPDATE
Объект проверки	Ключевой результат (`key_results`)
Важность	medium
Условие нарушения	Цель, к которой относится КР, имеет статус `active`, и `fact_updated_at < CURRENT_DATE - 28`.

Текст сообщения: «Факт по КР № {id} обновлялся {fact_updated_at}. С тех пор прошло больше 28 дней. Проверьте фактическое значение».

Пример: Факт по КР № 8 активной цели обновили 1 ноября. При проверке 29 ноября прошло ровно 28 дней, нарушения нет. Если 30 ноября дата обновления останется прежней, в списке проблем появится КР № 8.

```
SELECT kr.id
FROM key_results kr
JOIN goal g ON g.id = kr.goal_id
WHERE g.status = 'active'
  AND kr.fact_updated_at < CURRENT_DATE - 28;
```

3. Метрики качества данных

В каждом прогоне метрики рассчитываются для всех подразделений. При расчёте метрик каждого подразделения учитываются его собственные цели и КР, а также цели и КР всех его дочерних подразделений на любой глубине. Каждая цель или КР учитывается один раз в рамках соответствующей метрики. Значения метрик дочерних подразделений не усредняются.

Метрика 1 — Доля активных целей с проблемами

Показывает процент проблемных целей среди всех активных. Цель с несколькими проблемами учитывается один раз. Нарушения каскада относятся к завершённым и отменённым родительским целям и учитываются в метрике 3.

Поле	Значение
Формула	(число активных целей ветки, у которых в текущем прогоне есть ≥ 1 строка `problem` по самой цели или по её КР) ÷ (число активных целей ветки) × 100
Что входит в числитель	Активные цели учитываются по уникальным `problem.id_goal` текущего прогона. Для нарушения по КР это поле содержит `key_results.goal_id`. Три устаревших КР одной цели дают одну проблемную цель в метрике 1 и три проблемных КР в метрике 2.
Единица	%
Лучше	снижение (к 0%)
Дочерние подразделения	В метрику подразделения входят активные цели выбранного подразделения и всех его дочерних подразделений вниз по дереву (`division.parent_id`)
Нет активных целей	`null`

Пример: 10 активных целей в ветке, у 3 есть проблемы, получаем **30%**.

Метрика 2 — Доля ключевых результатов с устаревшим фактом

Показывает процент КР активных целей, у которых фактическое значение не обновлялось более 28 дней.

Поле	Значение
Формула	(число КР активных целей ветки с `problem.code = 'KR_NO_UPDATE'` в текущем прогоне) ÷ (число КР активных целей ветки) × 100
Источник	`problem` текущего прогона с `code = 'KR_NO_UPDATE'`; знаменатель — все `key_results` целей ветки со `status = 'active'`
Единица	%
Лучше	снижение (к 0%)
Дочерние подразделения	В метрику подразделения входят КР активных целей выбранного подразделения и всех его дочерних подразделений вниз по дереву (`division.parent_id`)
Нет КР у активных целей	`null`

Пример: В ветке у активных целей 20 КР, у 4 из них сработало `KR_NO_UPDATE`. Доля равна **4 ÷ 20 × 100 = 20%**.

Метрика 3 — Процент закрытых целей с активными потомками

Показывает, у какой части завершённых и отменённых родительских целей осталась хотя бы одна активная цель ниже по иерархии. Учитываются потомки на любой глубине.

Поле	Значение
Формула	Число родительских целей ветки подразделений с проблемой `GOAL_STATUS_CONFLICT` в текущем прогоне ÷ число завершённых и отменённых родительских целей в той же ветке на момент прогона × 100

Поле	Значение
Родительская цель	Цель, у которой есть хотя бы одна непосредственная дочерняя цель. Каждая родительская цель учитывается один раз.
Единица	%
Лучше	снижение (к 0%)
Дочерние подразделения	В метрику подразделения входят родительские цели выбранного подразделения и всех его дочерних подразделений вниз по дереву (`division.parent_id`). Потомки каждой такой цели по каскаду проверяются независимо от их подразделения
Нет завершённых или отменённых родительских целей	`null`

Пример: В ветке подразделений 10 завершённых или отменённых родительских целей. У двух из них остались активные потомки. Доля родительских целей с нарушением каскада — **2 из 10, или 20%**.

4. Сравнение с предыдущим прогоном

Как сопоставляются проблемы

Для проблемы по цели сравниваются `code` и `id_goal`, для проблемы по КР — `code` и `id_key_result`. Если ключ совпадает, это одна и та же проблема в двух прогонах.

Например, если в прогонах № 3 и № 5 есть запись с `code = GOAL_STATUS_STALE` и `id_goal = 11`, при сравнении этих прогонов проблема получает статус `same`. Проблема, которая появилась только в текущем прогоне, получает статус `new`. Если она была только в предыдущем, её статус — `resolved`.

Первый успешный прогон

Страница показывает результаты проверки и пояснение «Это первый успешный прогон». Блок сравнения скрыт.

Если предыдущий прогон завершился с ошибкой

Текущий прогон сравнивается с последним успешным прогоном перед ним. Например, если прогоны № 3 и № 5 успешны, а прогон № 4 завершился с ошибкой, результаты № 5 сравниваются с № 3. Прогон № 4 остаётся в истории со статусом `error`.

Как рассчитывается дельта метрик

Для каждой метрики одного подразделения API берёт из `run_metric` значения текущего и предыдущего успешных прогонов. Разница `diff = current − previous` выражается в процентных пунктах. Например, снижение с 40% до 30% даёт `diff = -10`. Если предыдущего успешного прогона нет, `previous` и `diff` равны `null`.

5. Контракт API страницы ККД

Базовый путь: `/api/v1/dq`. Все идентификаторы — целые числа. Имена полей записываются в формате `snake_case`.

Страница получает все три метрики выбранного прогона одним запросом. Список проблем загружается отдельно с учётом выбранных фильтров.

Общий формат ошибки

```
{
  "error": {
    "type": "invalid_request_error",
    "code": "resource_missing",
    "message": "Прогон 99 не найден",
    "param": "run_id"
  }
}
```

Коды ответов API

HTTP	code	Когда	Методы
200	—	Запрос успешно обработан, включая ответ с пустым списком	Все
400	`invalid_parameter`	Не передан обязательный параметр, неверный тип или недопустимое значение	Все
404	`resource_missing`	Запрошенный прогон или подразделение не найден	Статус прогона, метрики, проблемы
409	`run_not_finished`	Результаты пока недоступны, прогон выполняется (`running`)	Метрики, проблемы
409	`run_failed`	Результаты недоступны, прогон завершился с ошибкой (`error`)	Метрики, проблемы
500	`internal_error`	Внутренняя ошибка обработки запроса	Все

Поля ответа с ошибкой

Поле	Описание
`error.type`	Категория ошибки. Для HTTP 400, 404 и 409 — `invalid_request_error`. Для HTTP 500 — `api_error`.
`error.code`	Код ошибки из таблицы кодов ответов API.
`error.message`	Понятное пользователю описание ошибки. Для HTTP 500 — общее сообщение о внутренней ошибке.
`error.param`	Имя параметра, с которым связана ошибка. Для ошибок состояния прогона — `run_id`. Для HTTP 500 — `null`.
`run_status`	Дополнительное поле верхнего уровня, возвращается только при HTTP 409. Значение `running` для `run_not_finished`, `error` для `run_failed`.

Пример ответа HTTP 409 для прогона с ошибкой:

```
{
  "error": {
    "type": "invalid_request_error",
    "code": "run_failed",
    "message": "Результаты недоступны: прогон завершился с ошибкой",
    "param": "run_id"
  },
  "run_status": "error"
}
```

Метод 1. Получить список прогонов

`GET /api/v1/dq/runs`

Возвращает список прогонов со статусами и датами запуска и завершения. Прогоны отсортированы от новых к старым.

Параметры (query):

Параметр	Тип	Обязательный	Описание
limit	integer	нет	Число записей, по умолчанию 10, максимум 50
status	string	нет	Фильтр: `running`, `success`, `error`

Структура ответа:

- `data` — массив прогонов. Элемент: `id`, `status`, `start_run`, `finish_run`
- `has_more` — `true`, если есть ещё записи

Пример запроса последнего успешного прогона:

```
GET /api/v1/dq/runs?status=success&limit=1
```

Пример ответа:

```
{
  "data": [
    {
      "id": 5,
      "status": "success",
      "start_run": "2026-11-03T17:55:00Z",
      "finish_run": "2026-11-03T18:00:00Z"
    }
  ],
  "has_more": true
}
```

Метод 2. Получить статус прогона

`GET /api/v1/dq/runs/{run_id}`

Возвращает статус прогона и даты его запуска и завершения. Пока проверка идёт, страница периодически запрашивает её статус. При успешном получении состояния метод возвращает HTTP

200 для любого статуса прогона, включая `running` и `error`. Запрос получает состояние существующего прогона.

Параметры:

Параметр	Где	Тип	Обязательный	Описание
run_id	path	integer	да	id прогона

Структура ответа:

- `id`
- `status`: `running` | `success` | `error`
- `start_run`
- `finish_run` — `null`, если ещё выполняется

Пример запроса:

```
GET /api/v1/dq/runs/5
```

Пример ответа:

```
{
  "id": 5,
  "status": "success",
  "start_run": "2026-11-03T17:55:00Z",
  "finish_run": "2026-11-03T18:00:00Z"
}
```

Метод 3. Получить метрики по подразделению

`GET /api/v1/dq/runs/{run_id}/metrics?division_id={division_id}`

Возвращает все метрики с дельтой к предыдущему успешному прогону.

Параметры:

Параметр	Где	Тип	Обязательный	Описание
run_id	path	integer	да	id прогона
division_id	query	integer	да	id подразделения; метрики считаются по ветке

Структура ответа:

- `division`: `id`, `name`
- `run`: `id`, `status`, `finish_run`
- `previous_run_id` — id предыдущего `success` для дельты; `null`, если сравнивать не с чем
- `metrics` — объект из трёх метрик; у каждой `current`, `previous`, `diff` (могут быть `null`):
  - `affected_goals_pct` — доля активных целей с проблемами (%)
  - `stale_kr_pct` — доля ключевых результатов активных целей с устаревшим фактом (%)
  - `cascade_conflict_pct` — доля завершённых и отменённых родительских целей с нарушением каскада (%)

Пример запроса:

```
GET /api/v1/dq/runs/5/metrics?division_id=2
```

Пример ответа:

```
{
  "division": {
    "id": 2,
    "name": "Блок продаж"
  },
  "run": {
    "id": 5,
    "status": "success",
    "finish_run": "2026-11-03T18:00:00Z"
  },
  "previous_run_id": 3,
  "metrics": {
    "affected_goals_pct": {
      "current": 30,
      "previous": 40,
      "diff": -10
    },
    "stale_kr_pct": {
      "current": 20,
      "previous": 35,
      "diff": -15
    },
    "cascade_conflict_pct": {
      "current": 10,
      "previous": 15,
      "diff": -5
    }
  }
}
```

Метод 4. Получить список проблем прогона

`GET /api/v1/dq/runs/{run_id}/problems?division_id={division_id}`

Возвращает найденные проблемы прогона по ветке подразделения и дельту к предыдущему успешному прогону (`new` / `same` / `resolved`).

Параметры:

Параметр	Где	Тип	Обязательный	Описание
run_id	path	integer	да	id прогона
division_id	query	integer	да	id подразделения; проблемы ветки
code	query	string	нет	фильтр по коду правила, например `GOAL_STATUS_STALE`
importance	query	string	нет	`high`, `medium` или `low`
delta_status	query	string	нет	`new`, `same` или `resolved`
limit	query	integer	нет	по умолчанию 100, максимум 100

Структура ответа:

- `run_id`
- `previous_run_id` — `null`, если дельты нет
- `problems` — проблемы текущего прогона (`delta_status`: `new` или `same`)
- `resolved` — проблемы прошлого `success`, которых нет в текущем; те же поля элемента, `delta_status = resolved`
- `has_more` — обрезан ли список `problems` по `limit`

Элемент `problems` / `resolved`:

- `id`
- `code` — код правила
- `importance`
- `message` — текст, сохранённый при прогоне
- `id_goal`, `goal_name`
- `id_key_result`, `key_result` — `null`, если проблема только по цели
- `delta_status`: `new` | `same` | `resolved`

Пример запроса:

```
GET /api/v1/dq/runs/5/problems?division_id=2&limit=100
```

Пример ответа:


```
{
  "run_id": 5,
  "previous_run_id": 3,
  "problems": [
    {
      "id": 101,
      "code": "KR_NO_UPDATE",
      "importance": "medium",
      "message": "Факт по КР № 8 обновлялся 2026-10-01. С тех пор прошло больше 28 дней. Проверьте
фактическое значение",
      "id_goal": 20,
      "goal_name": "Вырастить выручку",
      "id_key_result": 8,
      "key_result": "Сократить время обработки заявки",
      "delta_status": "new"
    }
  ],
  "resolved": [
    {
      "id": 88,
      "code": "GOAL_STATUS_STALE",
      "importance": "high",
      "message": "Срок цели истёк 2026-10-01. Проверьте её статус: отмените или завершите цель либо
перенесите срок",
      "id_goal": 11,
      "goal_name": "Ускорить онбординг",
      "id_key_result": null,
      "key_result": null,
      "delta_status": "resolved"
    }
  ],
  "has_more": false
}
```


Граничные случаи

В таблицах указаны коды ответа и значения полей для каждого метода.

Метод 1. Список прогонов

Случай	Поведение
Прогонов ещё не было	HTTP 200, `data = []`, `has_more = false`
Нет прогонов с запрошенным `status`	HTTP 200, `data = []`, `has_more = false`

Метод 2. Статус прогона

Случай	Поведение
Прогон ещё выполняется	HTTP 200, `status = running`, `finish_run = null`
Прогон завершился с ошибкой	HTTP 200, `status = error`, `finish_run` заполнен
Прогон не найден	HTTP 404, `code = resource_missing`

Метод 3. Метрики

Случай	Поведение
Подразделение без целей	HTTP 200. Все `current = null` (`affected_goals_pct`, `stale_kr_pct`, `cascade_conflict_pct`). При нулевом знаменателе процент не определён. `previous` / `diff` — по правилам дельты
Первый успешный прогон	HTTP 200, `previous_run_id = null`, у метрик `previous = null`, `diff = null`
Прогон ещё выполняется (`running`)	HTTP 409, `code = run_not_finished`, `run_status = running`
Прогон с ошибкой (`error`)	HTTP 409, `code = run_failed`, `run_status = error` — результатов качества нет

Метод 4. Список проблем

Случай	Поведение
Подразделение без целей	HTTP 200, `problems = []`, `resolved = []`
Цель без ключевых результатов	HTTP 200. Если цель активна — обычная запись в `problems` с `code = GOAL_BAD_WEIGHTS`, `id_key_result = null`, сообщение «У активной цели нет ключевых результатов»
Прогон ещё выполняется (`running`)	HTTP 409, `code = run_not_finished`, `run_status = running`. Список проблем будет доступен после успешного завершения прогона.
Прогон с ошибкой (`error`)	HTTP 409, `code = run_failed`, `run_status = error`
Первый успешный прогон	HTTP 200, `previous_run_id = null`, `resolved = []`, у всех проблем `delta_status = new`

Мотивационное письмо для участия в проекте.

Почему тебе интересен этот проект?

Мне интересен этот проект, потому что он как раз про системный анализ: не просто собрать пожелания, а довести их до модели данных, правил, метрик и контракта API, с которыми может работать разработка.

На предыдущем опыте я уже занималась аналитикой при внедрении и выборе корпоративных систем, но тогда задачи были проще, а требования к глубине проработки - ниже. За прошедшие годы подходы и инструменты ушли вперёд, а интерес к профессии не пропал. Сейчас я целенаправленно возвращаюсь в эту область: прошла программу по бизнес-системному анализу и хочу развиваться дальше именно как системный аналитик - на задачах, где нужно однозначно описать решение для команды.

Кейс ККД по целям и ключевым результатам мне откликается тем, что в нём соединяются понимание процесса и аккуратная спецификация для разработки. Это то направление, в котором я хочу расти.

Как ты видишь свою роль в команде, которая создаёт продукт?

В команде, которая создаёт продукт, я вижу себя системным аналитиком на стыке бизнеса и разработки: собираю и уточняю требования, перевожу их в понятные артефакты: модель данных, правила, метрики, описания API, и сопровождаю реализацию вопросами и приёмкой.

Понимаю, что при подключении к новому проекту нужно время, чтобы влиться в коллектив и процессы. На старте готова браться за задачи в рамках этой роли: от уточнения требований и оформления спецификаций до разбора граничных случаев с разработкой и тестированием, и постепенно расширять зону ответственности по мере погружения в продукт.

Сколько времени в неделю ты готов(а) уделять проекту и в течение какого периода?

Готова работать над проектом полный рабочий день. Другой формат занятости также готова обсудить с командой.

На период стажировки готова подключиться сразу и пройти её полностью. Если по итогам сотрудничество продолжится в формате постоянной работы, для меня это желаемый сценарий, так как рассматриваю участие не как короткий эпизод, а как возможность закрепиться в команде и развиваться в роли системного аналитика.